

14/8/20
Ex No. 6 DESIGN & IMPLEMENT 2 bit MAGNITUDE
COMPARATOR USING XILINX NIVADO

Aim: To design & implement 2 bit magnitude comparator using verilog HDL & verify its operation using a testbench.

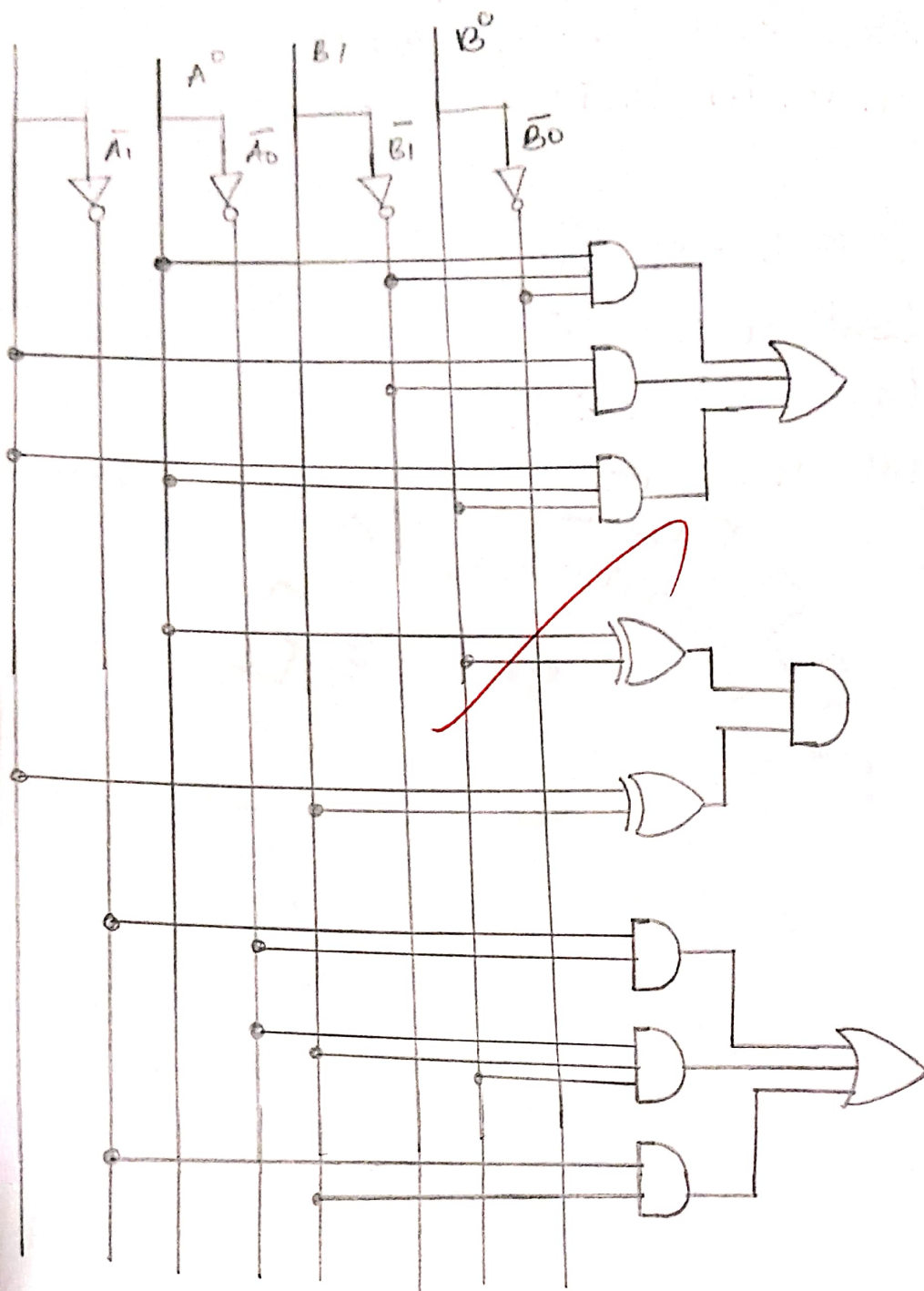
Components:

- Xilinx Nivado
- Verilog HDL
- Laptop

Code:

```
module comparator_2bit (  
    input [1:0] a,  
    input [1:0] b,  
    output reg eq,  
    output reg lt,  
    output reg gt,  
);  
always@ (a or b)  
begin  
    if (a == b)  
        begin  
            eq = 1'b1;  
            lt = 1'b0;  
            gt = 1'b0;  
        end  
    else if (a > b)  
        begin  
            eq = 1'b0;  
            lt = 1'b0;
```

O/P
Verified



end
else
begin
eq = 1'b0;
lt = 1'b1;
gt = 1'b0;
end
end
endmodule



Result :

The 2 bit magnitude comparator was successfully designed & implemented using verilog HDL.

Sum

18/8/26

Ex No. 4

Simulate & Verify 16 bit ALU using verilog

Aim:

To simulate & verify the operation of 16 bit ALU using verilog.

Apparatus required: Jilinas system.

Procedure:

- i) verilog language was used to code given module.
- ii) corresponding i/p & clock signal given.
- iii) Program compiled for the error.
- iv) Then it is simulated for given data.
- v) The O/P waveform is obtained.

Programming:

```
module alu (clk, rst, a, b, opcode);
```

```
input clk;
```

```
input rst;
```

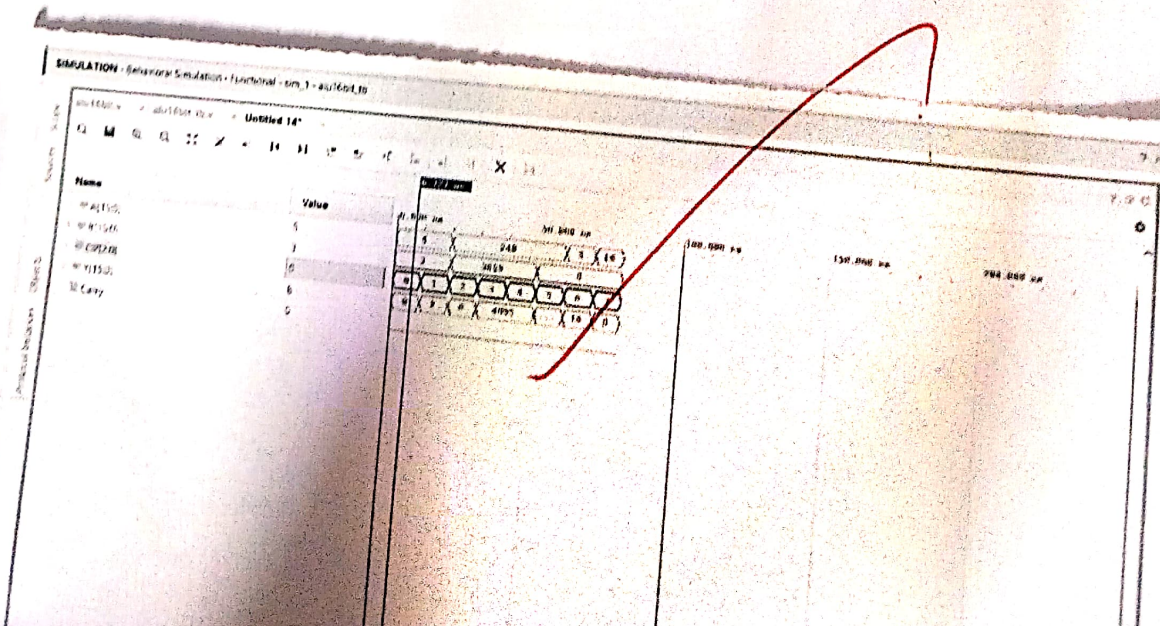
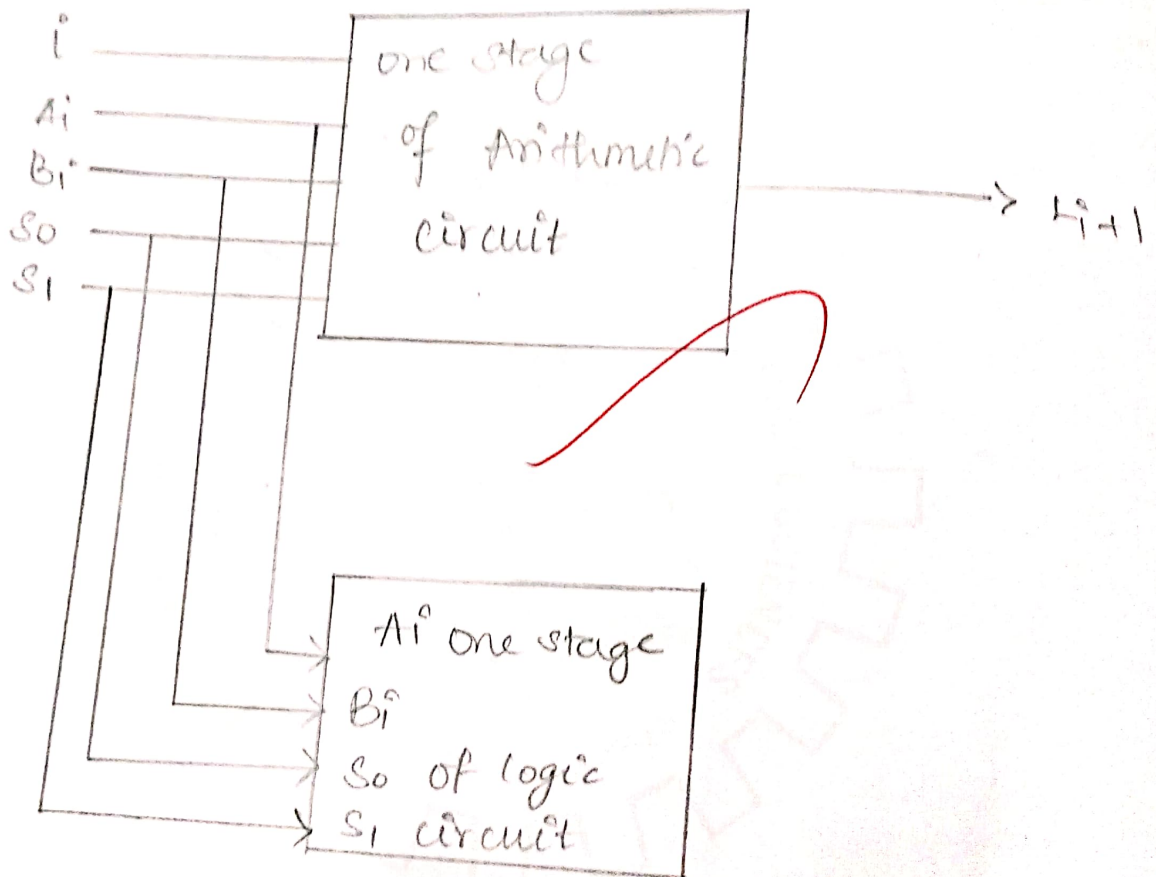
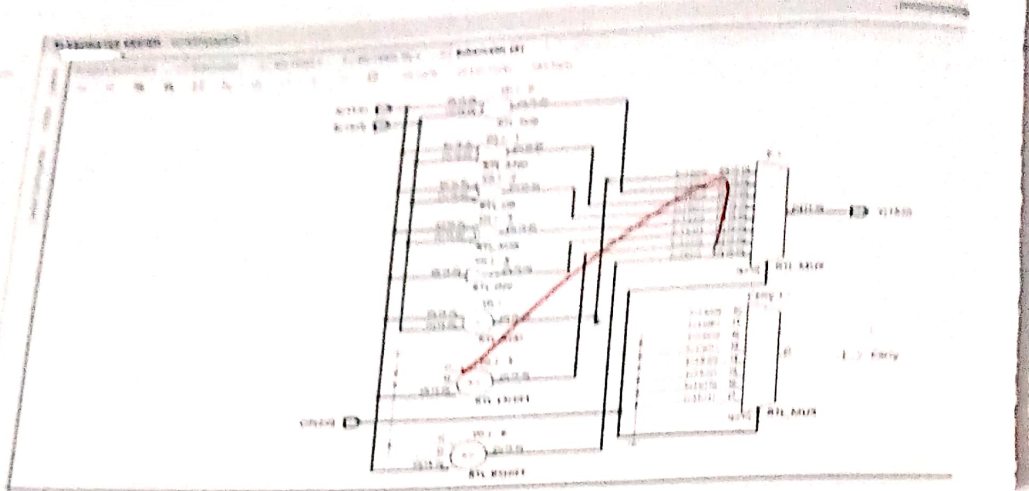
```
input [15:0] a, b;
```

```
input [2:0] opcode;
```

```
reg [15:0] ;
```

```
always @ (posedge clk (or) negedge rst)
```

```
begin
```



Observation:

S_2	S_1	S_0	function
0	0	0	$A + C_1^0$
0	0	1	$A + B + C_1^0$
0	1	0	$A + \bar{B} + C_1^0$
0	1	1	$A + 1 + C_1^0$
1	0	0	A AND B
1	0	1	A (OR) B
1	1	0	A (XOR) B
1	1	1	NOT A

if (~rst)

z <= 16'h00;

else,

case (opcode)

3'd0 : z <= a + b;

3'd1 : z <= a - b;

3'd2 : z <= a * b;

3'd3 : z <= a / b;

3'd4 : z <= a | b;

3'd5 : z = a ^ b;

3'd6 : z <= ~a;

endcase;

end

endmodule

Result:

Thus the operation was verified with the resultant waveform.

18/8/26

Ex No. 8

Design & Implement Multiplier using Xilinx Vivado

Aim: To implement a 4 bit multiplier using verilog HDL.

Components required. Xilinx System.

Procedure:

- 1) verilog language was used to code the given module.
- 2) corresponding i/p & clock signal given.
- 3) Program compiled for the error.
- 4) Then it is simulated for given data.
- 5) corresponding o/p waveform obtained & verified.

Program:

```
Module multiplier [a, b, p];  
input [3:0] a, b;
```

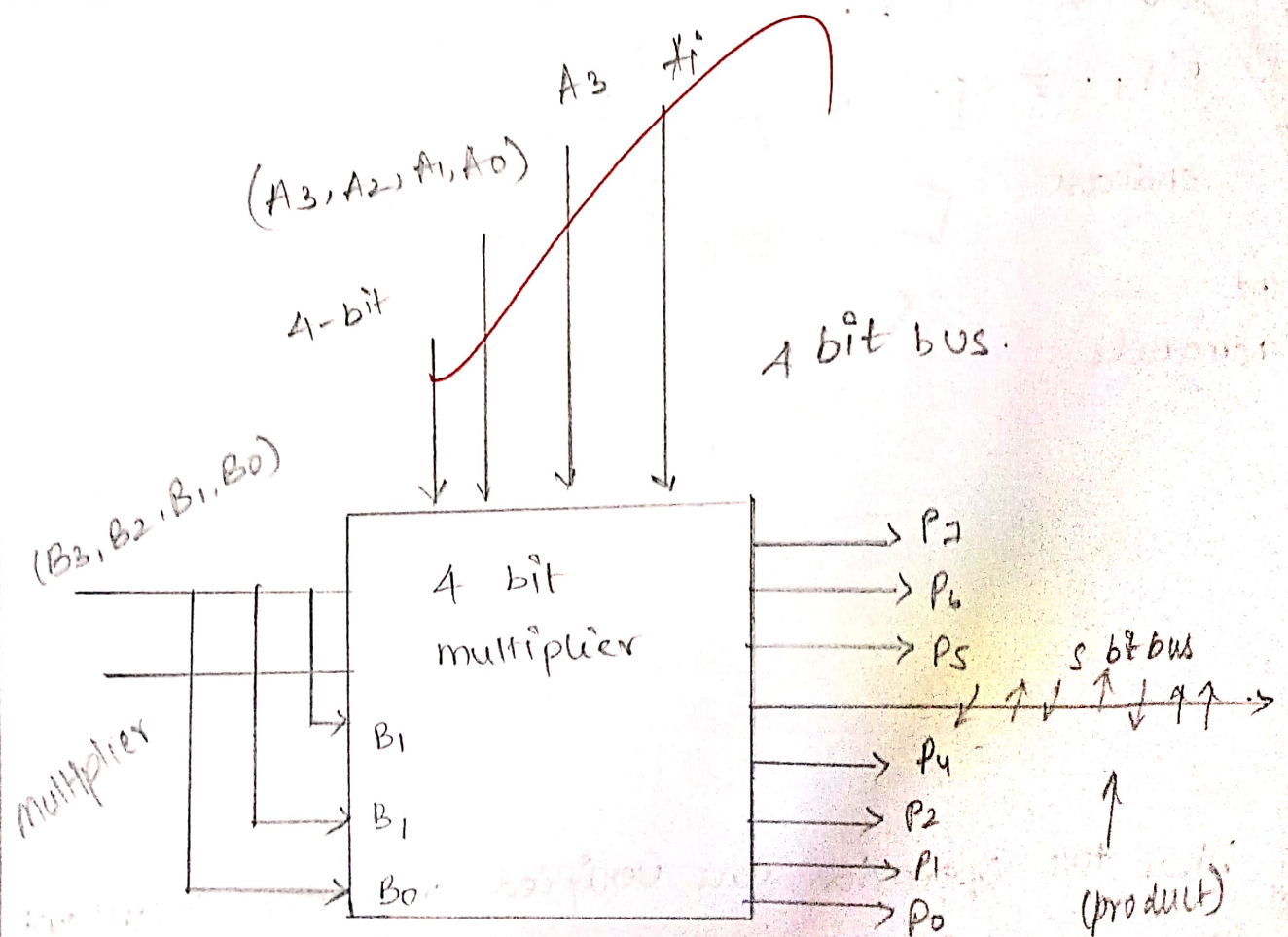
```
output [7:0] p;
```

```
assign p = a * b;
```

```
end module
```

~~o/p~~
~~verify~~

A (Binary)	B (Binary)	Decimal (A×B)	Product
0000	0000	0	00000000
0001	0001	1	00000001
0010	0011	6	00000110
0011	0100	12	00011000
0101	0010	10	00001010
0110	0011	18	00010010
1000	0010	16	00010000



Result:

Thus the 4 bit multiplier was successfully designed using Verilog HDL, simulated using Xilinx Vivado, & OIP waveform verified successfully.

Imp

17/8/26
Ex No. 9

40

Design & Implement flip flops using Jilinx Vivado

Aim: To design & implement flip flops using verilog HDL & verify their operation.

Apparatus required:

Jilinx Vivado, system

Procedure:

- ① Open Jilinx Vivado & create a new project.
- ② Write verilog code for required flip flop.
- ③ Create testbench.
- ④ Compile design & check syntax errors.
- ⑤ Simulate design
- ⑥ Verify o/p waveform.

A. JK

```
module JK_ff (clk, j, k, q)
```

```
input (clk, j, k);
```

```
output reg q;
```

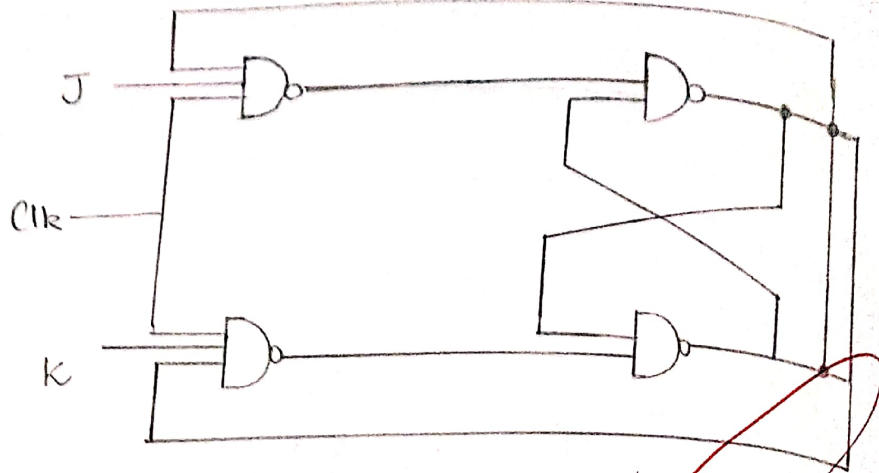
```
always @ (posedge clk)
```

```
begin
```

```
if (j == 1'b0 & k == 1'b0)
```

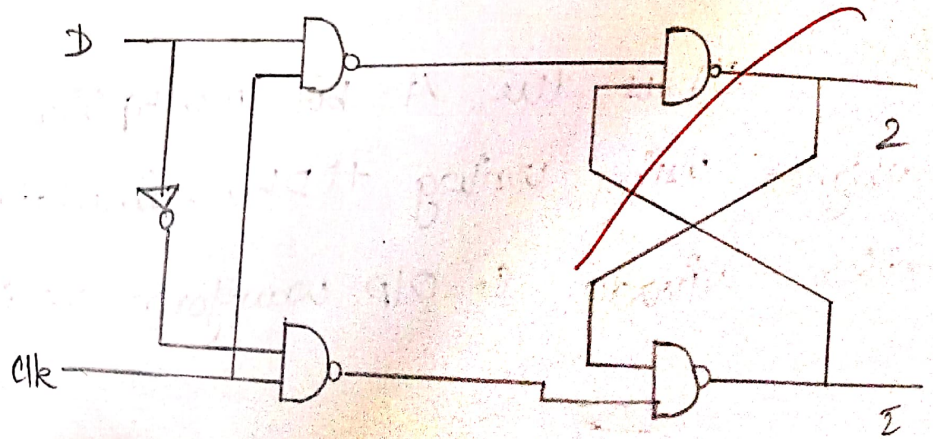
o/p waveform

① JK flip flop



J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

② D flip flop



```

q <= q;
else if (j == 1'b0 & k == 1'b1)
q <= 1'b0;
else if (j == 1'b1 & k == 1'b0)
q <= 1'b1;
else
q <= ~q;
end
end module

```

B. D flip flop

```

module d-ff (clk, d, q);
input clk, d;
output reg q;
always @ (posedge clk)
begin
q <= d;
end
endmodule

```

C. T flip flop :-

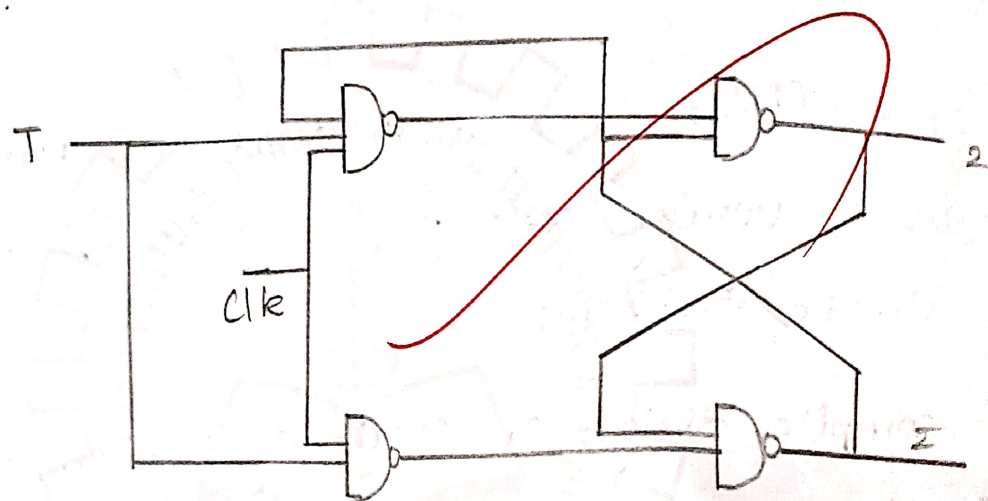
```

module t-ff (clk, t, q);
input clk, t;

```

Q	D	$Q(t+1)$
0	0	0
0	1	1
1	0	0
1	1	1

i) T flip flop:



T	Q_N	Q_{N+1}
0	0	0
0	1	1
1	0	0
1	1	0

q <= q;

else if (j == 1'b0 & k == 1'b1)

q <= 1'b0;

else if (j == 1'b1 & k == 1'b0)

q <= 1'b1

else

q <= not q;

end

end module

B. D flip flop

module d-ff (clk, d, q);

input clk, d

output reg q;

always @ (posedge clk)

begin

q <= d;

end

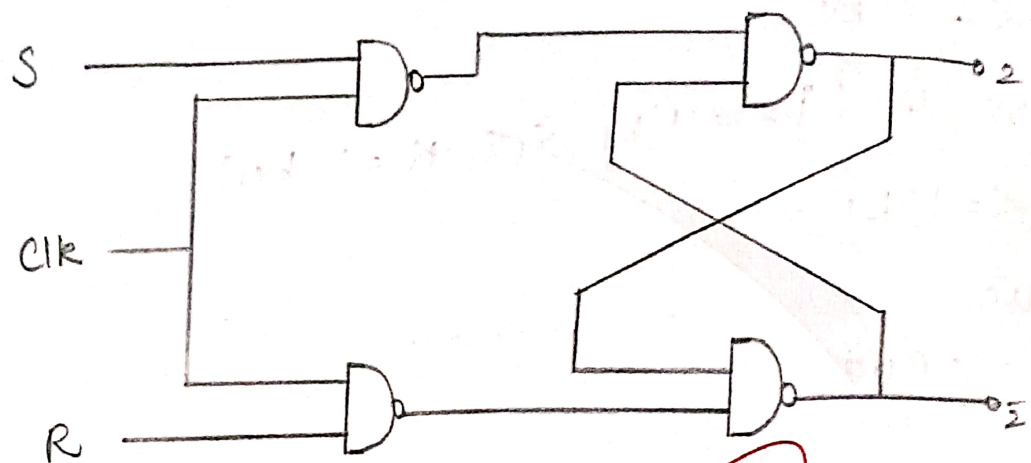
endmodule

C. T flip flop :-

module t-ff (clk, t, q);

input clk, t;

iv) SR flip flop.



S	R	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
1	0	1	0
1	0	0	1
1	0	1	1
1	1	0	1

output reg q;

always @ (posedge clk)

begin

if (t == 1'b0)

q <= q;

else

q <= ~q;

end

endmodule

iv) SR

module sr_ff (clk, s, r, q),

input clk, s, r;

output reg q;

always @ (posedge clk)

begin

if (s == 1'b0 & r == 1'b0)

q <= 1'b0

end

endmodule.

Result :

Thus JK, D & T flip flop were successful

designed using verilog HDL simulated
using Vivado.

Design & Implement Synchronous Counter using Xilinx Vivado

Aim:

To design & implement a synchronous counter using Verilog HDL & verify its operation.

Apparatus required:

Xilinx Vivado, system

Procedure:

- 1) Open Xilinx Vivado & create a new project.
- 2) write Verilog code for the synchronous counter.
- 3) Create a test bench.
- 4) simulate design.
- 5) Verify output waveform.

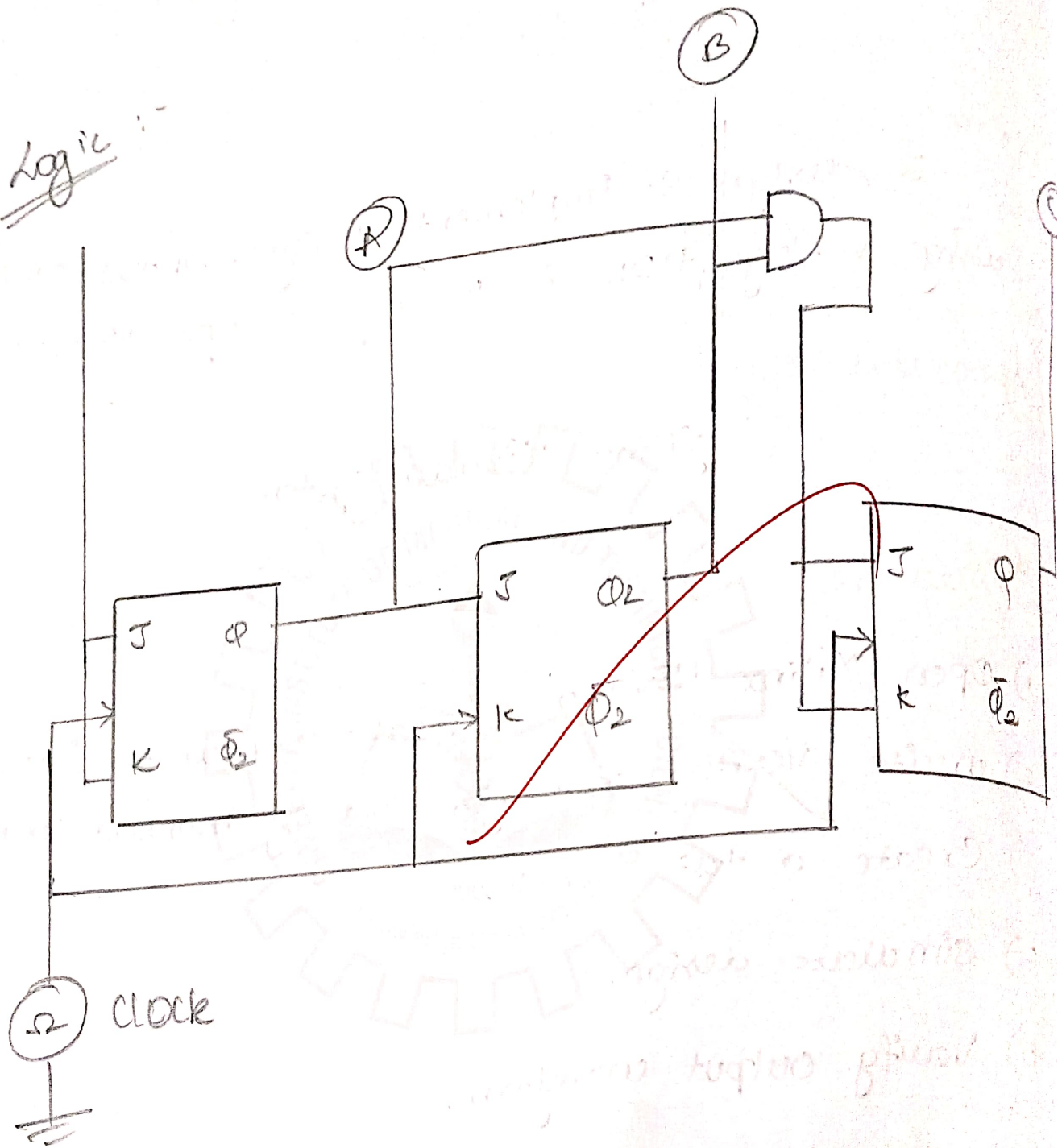
Program:

```
Module sync-counter (clk, reset, q);
```

```
input clk, reset;
```

```
output reg [3:0] q;
```

Logic :-



40

```
always @ (posedge clk)
begin
if (reset)
q = 4'b0000;
else
q <= q + 1'b1;
end
endmodule
```

Result:

Thus, the synchronous counter was successfully
designed using HDL, simulated in Xilinx Vivado.

Design & Implement Asynchronous Counter using Xilinx Nivado

Aim:

To design & implement an asynchronous counter using verilog HDL.

Apparatus required:

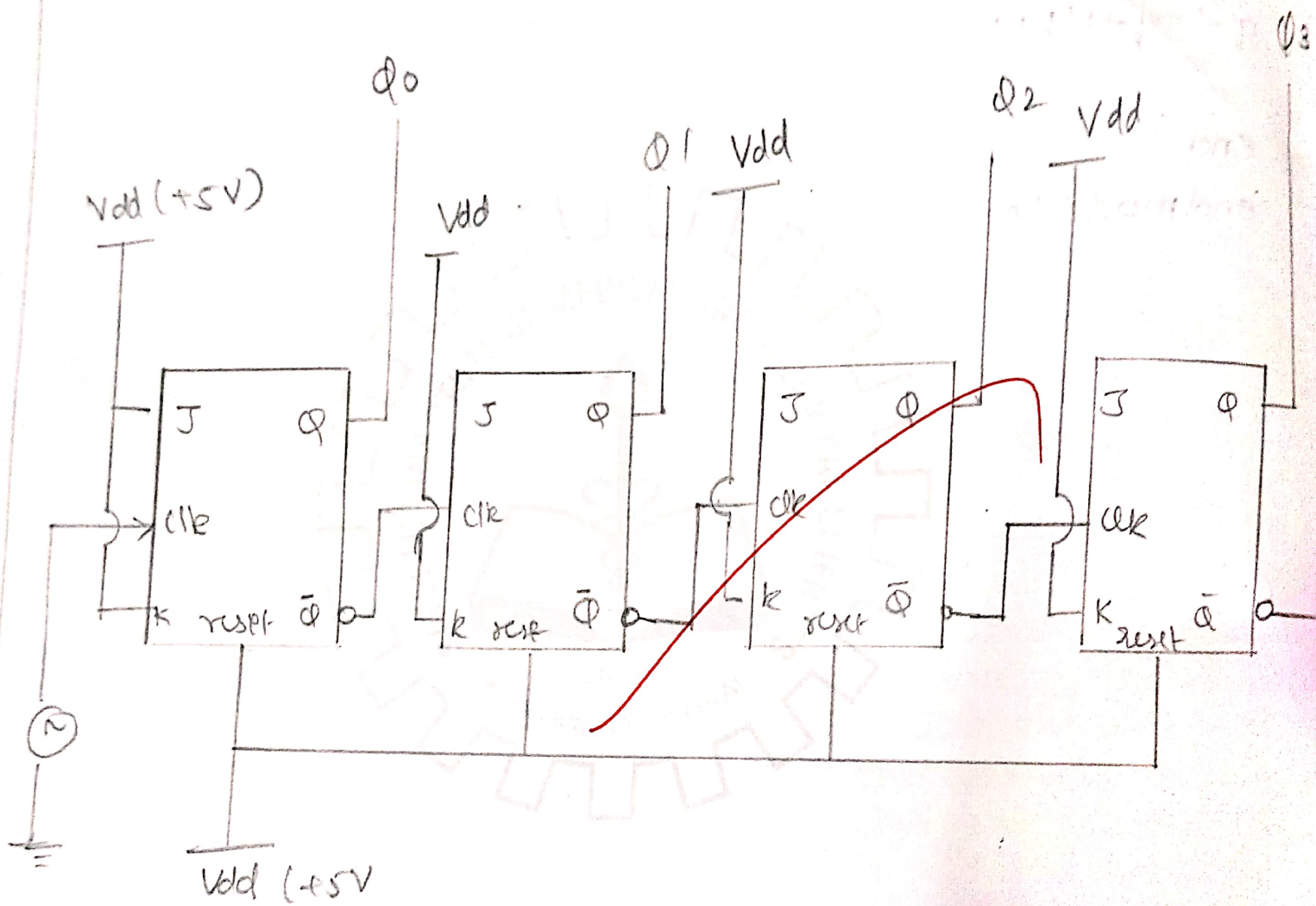
Xilinx Nivado, System

Procedure:

- ① Open Xilinx Nivado & create a new project.
- ② Write verilog code of the asynchronous counter.
- ③ Create the corresponding testbench.
- ④ Simulate the design.
- ⑤ Verify the o/p waveform.

Program :-

```
Module asynch_counter (clk, reset, q);  
input clk, reset;
```



output reg q;
always @ (posedge clk (or) posedge reset)
begin
if (reset)
q <= 16'd0;
end
endmodule

Result:

Thus the asynchronous counter was successfully designed using verilog HDL. Simulate in Xilinx vivado & o/p was verified successfully.

Ex No. 12

Design & implement traffic light controller using Vivado

Aim: To design & implement a traffic light controller using Xilinx Verilog HDL in Vivado & verify its operation through simulation.

Apparatus required :-

Xilinx system.

Procedure :-

- ① Open xilinx Vivado & create a project.
- ② write the verilog code for required flip flop.
- ③ Create testbench.
- ④ Compile the design & check for syntax errors.
- ⑤ simulate the design & verify the waveform.

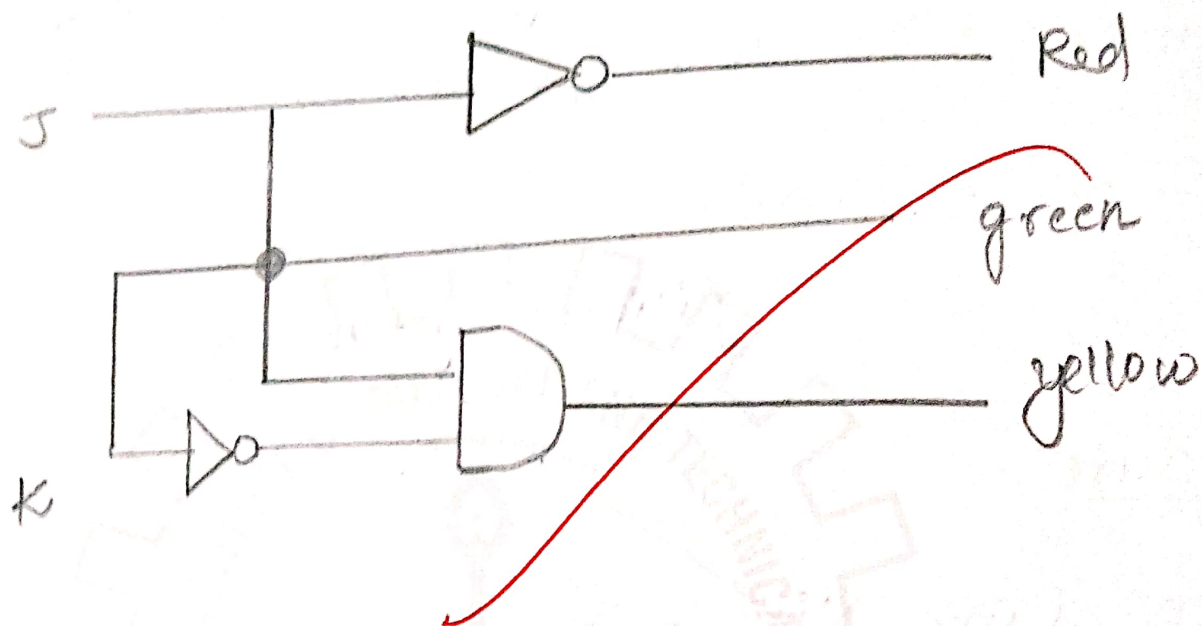
Code :-

Module tb-traffic light controller;

reg clk;

reg rst;

wire red;



wire green;

traffic - light - controller (

clk (clk);

rst (rst);

Red (Red);

yellow (yellow);

green (Green);

);

always # clk = ~clk;

initial

begin

clk = 0;

rst = 1;

10 rst = 0;

60;

\$ finish;

end

endmodule

Result :

Thus, the traffic light controller using Vivado is successfully simulated & verified.

To design & implement a digital clock using verilog HDL in Xilinx Vivado

Aim:

To design & implement a digital clock using verilog HDL in Xilinx Vivado.

Apparatus required:

System, Xilinx Vivado

Procedure:

- i) Open Xilinx Vivado
- ii) Create a new RTL project.
- iii) Add the verilog source file.
- iv) Add the testbench file.
- v) Simulate & verify the o/p waveform.

Code:-

verilog code

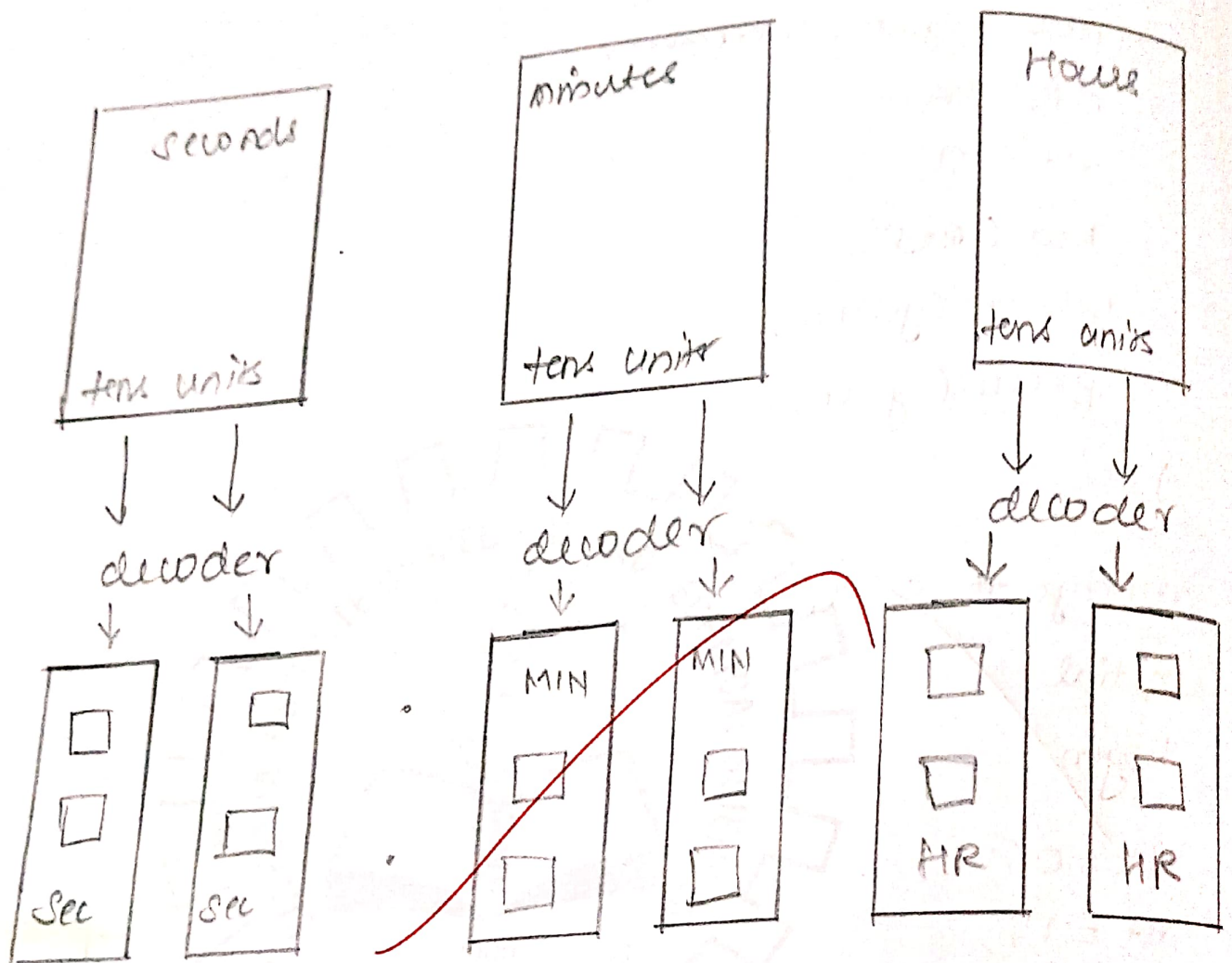
module digital-clock(

input clk

input rst,

output reg [5:0] sec,

);



always @(posedge clk (or) posedge rst);

begin

if (rst);

sec <= 0;

sec <= 0;

else

sec <= sec + 1;

end

end module

Timescale - 1ns / xps

module tb-digital-clock;

reg clk, rst;

wire [5:0] sec;

digital-clock ust(

clk, clk;

rst (rst);

sec (sec);

);

always # 5 clk ~ clk;

initial design

clk = 0;

rst = 1;

10 rst = 0

760

\$ finish

end

endmodule

Result :

The digital clock was successfully designed
by simulated in Xilinx Vivado.

design and simulate CMOS Inverter using cadence design flow.

Aim

to design and simulate the CMOS Inverter using cadence virtuoso and analysis its transistor characteristics and transient response.

Objectives

to design CMOS inverter schematic

to perform DC Sweep analysis

to perform transient analysis

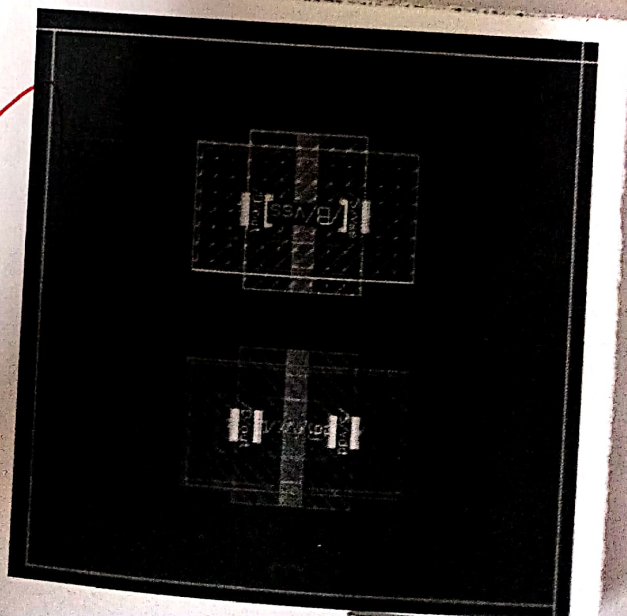
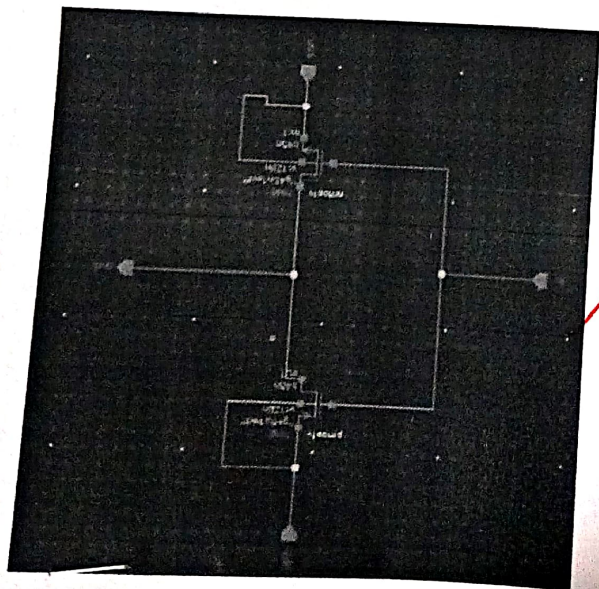
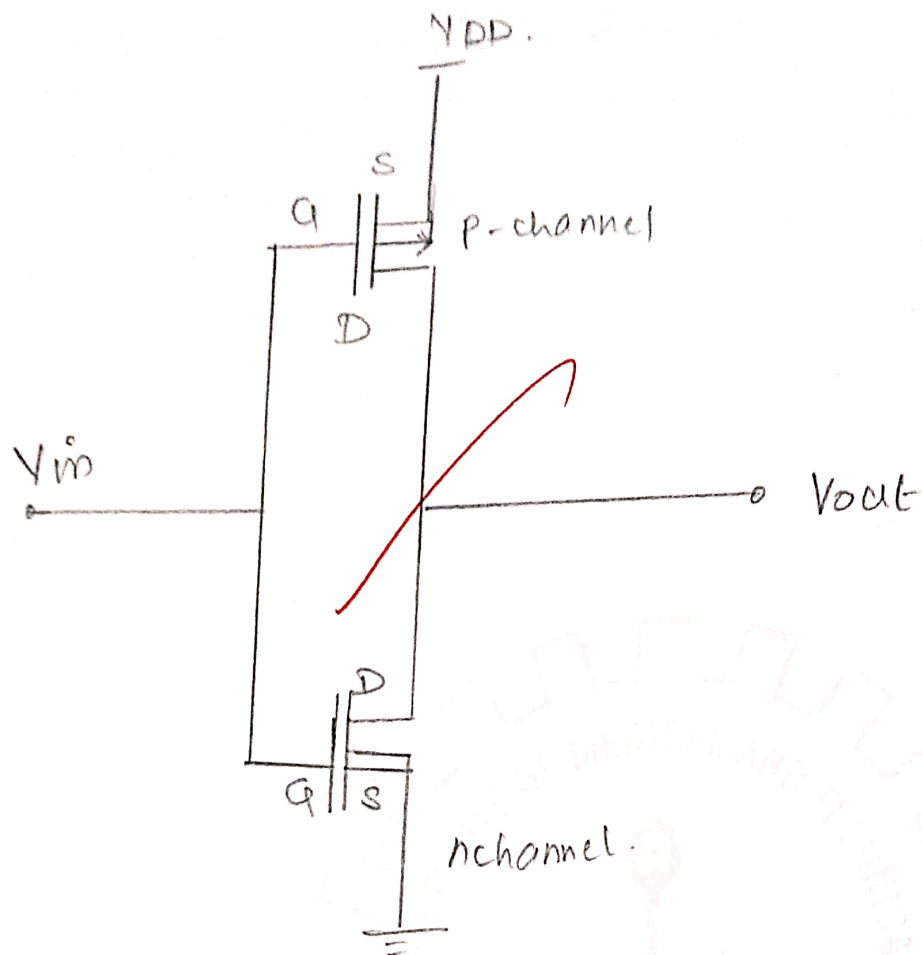
to determine switching threshold & propagation delay.

Software and tool used

cadence virtuoso

Spectre simulator

CMOS technology library



270

procedure:

open cadence virtuoso
create a new library and attach uios file
create new cell view (schematic)
place, PMOS to VDD, NMOS to GND and gates together
as v_{in} , drains together as v_{out} .

Add DC voltage source

check and save

open ADE

select DC sweep analysis

sweep v_{in} from 0V to VDD

plot v_{out} vs v_{in}

Replace DC input with pulse source

select transient analysis

Run simulation.

plot v_{in} and v_{out} .

calculation:

propagation delay

$$t_p = (t_p(1) + t_p(1)) / 2$$

~~Result:~~

The CMOS inverter was successfully designed and simulated using cadence virtuoso, the dc and transient analysis were performed and the switching characteristics were verified.

EX. NO: 15

Design and DC of transient analysis of a CMOS all gates using cadence virtuoso.

Aim

To design and simulate a 2-input CMOS all gates using cadence virtuoso and analysis is DC and transient characteristics.

Objectives

To design of 2-input CMOS all gates schematic
To understand pull up and pull down network
To perform DC technology library.

Procedure:

open cadence virtuoso
create new library and attach CMOS technology
create a new schematic cell view place
2 PMOS and 2 NMOS transistor.
Add VDD and GND synthesis

Calculations:

$$t_p = (t_{pLH} + t_{pHL}) / 2$$



Handwritten signature
Result:

Handwritten signature
The CMOS NAND gate was successfully designed & simulated.